

UNIX

Rapid Revision Handbook

Everything you need the day before the exam —
commands, syntax, comparisons, MCQs & memory tricks

- ✓ Every command from the TCS IPA UNIX syllabus, explained concisely
 - ✓ Comparison tables for the most confused command pairs
- ✓ 120+ practice MCQs in TCS IPA style, with explained answers
 - ✓ A 6-page rapid-fire revision section for the final hours

How to Use This Book

This handbook replaces the original TCS IPA UNIX PDF for revision. Each chapter gives you the concept in a few lines, a command reference table, the exact examples you're expected to know, an exam tip, and a common mistake to avoid — no filler.

- **First pass (2–3 days before exam):** Read Chapters 1–15 fully, run the examples in a terminal if possible.
- **Second pass (day before exam):** Read only the shaded EXAM TIP / MISTAKE boxes and the Comparison Tables chapter.
- **Final hours:** Read only the “Rapid Revision” section at the end — it condenses the entire book to 6 pages.
- **Practice:** Attempt the 120+ MCQs in the Practice section; check answers only after attempting.

Table of Contents

UNIX Command Cheat Sheet	4
Introduction to UNIX	6
File System Hierarchy	7
Basic File Navigation	8
File Creation and Management	9
Viewing Files	10
Permissions	11
Filters (grep, find, sort, awk, sed & more)	12
Redirection and Pipes	14
Users and Group Management	15
Shells and Shell Types	16
Processes	17
Background Jobs and Job Control	18
Shell Scripting Basics	19
Conditional Statements	20
Loops in Shell Scripts	21
TCS IPA Important Points	22

Complete Command Summary	23
Comparison Tables	24
Practice MCQs	26
Rapid Revision — Final Hours	44

UNIX Command Cheat Sheet

One-page reference of the most-used commands, grouped by task.

Navigation & Files

Command	Purpose / Example
<code>pwd</code>	print working directory
<code>cd dir</code>	change directory (cd - : previous, cd ~ : home, cd .. : up one)
<code>ls -la</code>	list all files, long format, incl. hidden
<code>mkdir/rmdir</code>	make / remove empty directory
<code>touch f</code>	create empty file / update timestamp
<code>cp/mv/rm -r</code>	copy / move / delete recursively
<code>find . -name x</code>	search files by name/type/permission

Viewing

Command	Purpose / Example
<code>cat -n f</code>	show file with line numbers
<code>head/tail -n5 f</code>	first/last 5 lines
<code>tail -f f</code>	follow file live (logs)
<code>less/more f</code>	page through file

Filters & Text

Command	Purpose / Example
<code>grep -i -v -n -c -E</code>	case-insens / invert / line-no / count / regex
<code>sort -r -u -k2 -n</code>	reverse / unique / by column2 / numeric
<code>uniq / wc -l -w -c</code>	remove dup adjacent lines / count lines-words-chars
<code>cut -d',' -f1</code>	extract field 1, comma-delimited
<code>awk '{print \$2}'</code>	print column 2
<code>sed 's/a/b/'</code>	replace a with b (first match/line)
<code>tr / rev / tac</code>	translate chars / reverse chars / reverse line order

Permissions

Command	Purpose / Example
<code>chmod 755 f</code>	rxr-xr-x
<code>chmod u+x f</code>	add execute for owner
<code>chown user f</code>	change owner
<code>umask</code>	default permission mask

Processes & Jobs

Command	Purpose / Example
<code>ps aux / top</code>	list processes / live monitor

Command	Purpose / Example
kill PID	terminate a process
cmd &	run in background
jobs / fg / bg	list / foreground / background jobs
Ctrl+C / Ctrl+Z	kill process / suspend process

Users & Shell

Command	Purpose / Example
whoami / id / groups	current user / uid-gid / group list
sudo / su	run as root / switch user
passwd	change password
echo \$SHELL	show current shell

Redirection

Command	Purpose / Example
cmd > f	overwrite file with output
cmd >> f	append output
cmd < f	take input from file
cmd1 cmd2	pipe output to next command

Chapter 1 — Introduction to UNIX

Concept

- UNIX is a **multi-user, multitasking** OS developed in the 1970s at Bell Labs.
- Known for portability, stability, and a modular “small tools that work together” design.
- **Kernel** = core that talks to hardware; **Shell** = interface that interprets your commands.
- **Process** = a running program instance; **File** = a named stream of bytes — in UNIX, almost everything (devices, pipes, sockets) is treated as a file.
- Forms the base of Linux, macOS (Darwin), Solaris, AIX.

Command Reference

Command	Purpose / Syntax
<code>uname -a</code>	print system/kernel information
<code>uname -s / -r</code>	print OS name / kernel release only

Worked Examples

```
$ uname -a
Linux hostname 5.15.0 x86_64 GNU/Linux
```

★ *EXAM TIP: “Everything is a file” is a favourite conceptual MCQ line — remember it verbatim.*

■ **COMMON MISTAKE:** Don't confuse kernel (manages hardware/resources) with shell (interprets your typed commands). Kernel is invisible to the user; shell is what you interact with.

Chapter 2 — File System Hierarchy

Concept

- The filesystem is a single **tree** rooted at **/** — there are no separate drive letters like C:\ (unlike Windows).
- **/bin** essential binaries · **/etc** config files · **/home** (Linux) or **/Users** (macOS) user directories · **/usr** user programs/libraries · **/var** variable data (logs) · **/opt** optional/third-party software · **/dev** device files · **/tmp** temporary files.
- **Hidden files** start with a dot, e.g. **.bashrc**, and don't show in a plain **ls**.
- macOS uses **/Users** instead of Linux's **/home**, but both otherwise follow a very similar Unix-style tree.

Command Reference

Command	Purpose / Syntax
/	root of the entire filesystem tree
/bin, /sbin	essential user/system binaries
/etc	system-wide configuration files
/home (Linux) /Users (macOS)	personal user directories
/usr	installed programs, libraries, docs
/var	logs, mail, spool — data that changes often
/dev	device files, e.g. /dev/sda
/tmp	temporary files, cleared on reboot

Worked Examples

```
$ ls -a ~ # shows hidden dotfiles like .bashrc, .ssh
$ cat /etc/hosts # maps hostnames to IP addresses
```

★ **EXAM TIP:** PDF-style question: “Which directory holds config files?” → **/etc**. “Which holds device files?” → **/dev**.

■ **COMMON MISTAKE:** Don't say macOS has no root filesystem tree — it does; only the user-folder name (**/Users** vs **/home**) differs from Linux.

Chapter 3 — Basic File Navigation

Concept

- **Absolute path** starts from / (e.g. /home/user/file) — always the same regardless of where you are.
- **Relative path** is relative to your current directory (e.g. ../file, ./file).
- Wildcards: * matches any characters, ? matches one character, [abc] matches one of a/b/c.
- Tab completion auto-completes file/directory names — speeds up typing and avoids typos.

Command Reference

Command	Purpose / Syntax
<code>pwd</code>	print current working directory
<code>cd dir</code>	change into dir
<code>cd ..</code>	move up one directory
<code>cd -</code>	go to the previous directory
<code>cd ~ / cd</code>	go to home directory
<code>ls</code>	list directory contents
<code>ls -l</code>	long listing: permissions, owner, size, date
<code>ls -a</code>	include hidden (dotfile) entries
<code>ls -la</code>	combine both -l and -a

Worked Examples

```
$ cd /var/log # absolute path
$ cd ../docs # relative path
$ ls -l ~/Downloads
```

★ *EXAM TIP: `cd -` is a very common trick question — it toggles back to the PREVIOUS directory, not the parent.*

■ **COMMON MISTAKE:** `cd ..` moves UP one level (parent); it does NOT go to the home directory — that's `cd ~` or plain `cd`.

Chapter 4 — File Creation and Management

Concept

- **touch** creates an empty file or updates its modification timestamp if it already exists.
- **cp** copies; **mv** moves or renames (same command does both, depending on destination).
- **rm** deletes files; add **-r** for directories, **-f** to force without confirmation.
- **rm -rf** is permanent and irreversible — there is no Recycle Bin in the shell.

Command Reference

Command	Purpose / Syntax
touch f	create empty file / update timestamp
cp src dst	copy file(s)
cp -r src dst	copy directory recursively
mv src dst	move or rename
rm f	delete a file
rm -r dir	delete directory and contents
rm -rf dir	force-delete directory, no prompts
mkdir dir	create a new directory
rmdir dir	remove an EMPTY directory only

Worked Examples

```
$ cp notes.txt backup/ # copy into backup folder
$ rm -rf temp/ # permanently deletes temp/ and all contents
```

★ *EXAM TIP: rmdir only works on EMPTY directories — to remove a non-empty one, you need rm -r.*

■ **COMMON MISTAKE:** People often think mv makes a copy like cp — it does not; the source is gone after mv.

Chapter 5 — Viewing Files

Concept

- **cat** dumps the whole file at once — fine for small files, unusable for huge logs.
- **less** and **more** page through a file; **less** is more powerful (scroll both directions, search with /).
- **head/tail** show first/last 10 lines by default; use **-n** to change the count.
- **tail -f** “follows” a growing file live — the standard way to watch a log in real time.

Command Reference

Command	Purpose / Syntax
cat f	print entire file
cat -n f	print with line numbers
more f	page forward only, older tool
less f	page both ways, searchable, modern default
head f / head -n5 f	first 10 / first 5 lines
tail f / tail -n5 f	last 10 / last 5 lines
tail -f f	live-follow a growing file (e.g. logs)

Worked Examples

```
$ less /var/log/syslog
$ tail -f app.log # watch new log lines appear live
```

★ **EXAM TIP:** “Which command monitors a log file in real time?” → **tail -f**. This is one of the most repeated TCS questions.

■ **COMMON MISTAKE:** less vs more: less can scroll UP and DOWN and supports search; more can only go forward. Don't mix them up.

Chapter 6 — Permissions

Concept

- Every file has 3 permission sets: **User (owner)**, **Group**, **Others** — each with Read(r)/Write(w)/Execute(x).
- **Symbolic mode:** `chmod u+x, g-w, o=r, a+rx` (u/g/o/a = user/group/other/all; +/- = add/remove/set).
- **Octal mode:** `r=4, w=2, x=1` — add them per group. `7=rx`, `6=rw`, `5=r-x`, `4=r--`, `0=---`.
- **chown** changes the file's owner (and optionally group with `chown user:group`).
- **umask** is subtracted from the default max permissions to decide permissions of NEWLY created files.

Command Reference

Command	Purpose / Syntax
<code>chmod 755 f</code>	owner <code>rx</code> , group <code>r-x</code> , others <code>r-x</code>
<code>chmod 644 f</code>	owner <code>rw-</code> , group <code>r--</code> , others <code>r--</code> (typical for regular files)
<code>chmod 700 f</code>	owner <code>rx</code> , group/others no access
<code>chmod u+x f</code>	add execute permission for owner only
<code>chmod a+rx f</code>	give everyone full permissions
<code>chown user f</code>	change file owner to 'user'
<code>chown user:group f</code>	change owner and group together
<code>umask</code>	show/set default permission mask

Worked Examples

```
$ chmod 700 script.sh # only owner can read/write/execute
$ chown root config.txt # give ownership to root
$ chmod u+rx,g=rx,o-rwx emp[1-3] # symbolic mode on emp1, emp2, emp3
```

★ **EXAM TIP:** Memory trick for octal: `r=4, w=2, x=1` — sum what's granted. `7=4+2+1 (rx)`, `5=4+1 (r-x)`, `6=4+2 (rw-)`.

■ **COMMON MISTAKE:** `chmod 777` gives EVERYONE full access — a classic “worst practice” MCQ distractor; don't pick it as a “safe” answer.

Chapter 7 — Filters (grep, find, sort, awk, sed & more)

Concept

- **grep** searches for a text pattern in file(s), line by line, and supports regular expressions.
- **find** searches the DIRECTORY TREE for files/dirs by name, type, permission, size, etc. (different job from grep, which searches file CONTENT).
- **sort** orders lines; **uniq** removes ADJACENT duplicate lines (usually used after sort).
- **cut** extracts columns/fields by delimiter; **awk** is a full field-processing language (\$1, \$2... are columns, NR = row count).
- **sed** does stream editing, most often find-and-replace: sed 's/old/new/'.
- **wc** counts lines/words/characters; **rev** reverses characters in a line; **tac** reverses the ORDER of lines/files; **tr** translates/deletes individual characters.

Command Reference

Command	Purpose / Syntax
grep 'pat' f	print lines matching pattern
grep -i	case-insensitive match
grep -v	invert match — print non-matching lines
grep -c	count matching lines (not occurrences)
grep -n	show line numbers
grep -o	print only the matched text, not the whole line
grep -E	extended regex, e.g. {n,}, , +
grep '^pat'	match pattern at START of line
grep 'pat\$'	match pattern at END of line
find . -type f	find regular files only (-type d for directories)
find . -perm 0777	find files with exact permission 777
wc -l -w -c f	count lines / words / characters
sort f / sort -r f	sort ascending / descending
sort -u f	sort and remove duplicate lines
sort -k2 -n f	sort numerically by 2nd column
uniq f	remove adjacent duplicate lines
cut -d',' -f1 f	extract field 1 using comma delimiter
awk '{print \$2}' f	print the 2nd column of every line
sed 's/a/b/' f	replace first 'a' with 'b' on each line

Worked Examples

```
$ grep -o -i "unix" myfile | wc -l # count occurrences of 'unix', case-insensitive
$ grep -E '\w{6,}' file # words with 6+ letters
$ awk 'BEGIN{FS="|"; s=0} {s=s+$2} END{print s/NR}' file1 # average of column 2, '|' delimited
```

```
$ cut -d',' -f1 users.csv | sort | uniq # unique first-field values
```

★ *EXAM TIP: grep searches CONTENT of files; find searches the FILE SYSTEM for files matching name/type/permission. This exact distinction is asked directly.*

■ **COMMON MISTAKE:** grep -c counts MATCHING LINES, not the number of times the pattern occurs on a line — use grep -o | wc -l to count total occurrences.

Chapter 8 — Redirection and Pipes

Concept

- `>` redirects output to a file, **OVERWRITING** it. `>>` appends instead of overwriting.
- `<` feeds a file's content as input to a command.
- `|` (pipe) sends the **OUTPUT** of one command directly as **INPUT** to the next — chains commands without temp files.
- These are foundational to building one-liner data pipelines, e.g. `cat f | grep x | sort | uniq -c`.

Command Reference

Command	Purpose / Syntax
<code>></code>	redirect stdout to file, overwrite
<code>>></code>	redirect stdout to file, append
<code><</code>	redirect a file as stdin
<code> </code>	pipe stdout of cmd1 into stdin of cmd2

Worked Examples

```
$ echo "Hello" > greet.txt # overwrite greet.txt with 'Hello'
$ echo "Again" >> greet.txt # append 'Again' to greet.txt
$ cat file.txt | grep "name" # pipe file content into grep
```

★ **EXAM TIP:** `>` vs `>>` is a guaranteed MCQ: `>` destroys existing content, `>>` preserves it and adds to the end.

■ **COMMON MISTAKE:** Don't confuse `<` (read **FROM** a file as input) with `>` (write output **TO** a file) — the arrow points in the direction data flows.

Chapter 9 — Users and Group Management

Concept

- **whoami** shows the current logged-in username; **id** shows uid/gid; **groups** lists groups a user belongs to.
- **useradd/adduser** create new users (adduser is often a friendlier wrapper on Debian/Ubuntu).
- **passwd** sets/changes a password; **usermod** modifies an existing user's settings.
- **sudo** runs a single command with elevated (root) privileges; **su** switches to another user's full session.

Command Reference

Command	Purpose / Syntax
whoami	print current username
id	print uid, gid, and group memberships
groups [user]	list groups a user belongs to
useradd / adduser	create a new user account
passwd	set or change a password
usermod	modify an existing user's account
su	switch to another user
sudo cmd	run one command as root/superuser

Worked Examples

```
$ sudo adduser devuser # create a new user with elevated rights
$ groups devuser # list groups devuser belongs to
```

★ *EXAM TIP: su switches your WHOLE session to another user; sudo elevates privileges for just ONE command — a very common comparison question.*

■ **COMMON MISTAKE:** Don't confuse useradd/adduser (creating a user) with usermod (modifying an EXISTING user) — usermod cannot create new accounts.

Chapter 10 — Shells and Shell Types

Concept

- The **shell** is the command-line interface between the user and the kernel — it reads and executes your commands.
- Common shells: **sh** (original Bourne shell), **bash** (Bourne Again Shell, most widely used), **zsh** (macOS default since Catalina), **csh** (C-shell, syntax resembles C).
- **echo \$SHELL** reveals which shell you're currently running.
- The **shebang** line (**#!/bin/bash** or the more portable **#!/usr/bin/env bash**) at the top of a script tells the OS which interpreter to use to run it.

Command Reference

Command	Purpose / Syntax
echo \$SHELL	print path of the current shell
bash script.sh	run a script explicitly with bash
#!/bin/bash	shebang: hard-codes the bash interpreter path
#!/usr/bin/env bash	shebang: finds bash via PATH — more portable across systems

Worked Examples

```
$ echo $SHELL
/bin/bash
$ chmod +x script.sh && ./script.sh
```

★ **EXAM TIP:** PDF/exam answer expects **#!/bin/bash** as the standard shebang — note in modern practice **#!/usr/bin/env bash** is preferred for portability, but for TCS IPA, remember the PDF's **#!/bin/bash** form.

■ **COMMON MISTAKE:** A script needs BOTH a shebang line AND execute permission (**chmod +x**) to run directly as **./script.sh**.

Chapter 11 — Processes

Concept

- A **process** is a running instance of a program; each gets a unique **PID** (Process ID).
- **ps** gives a snapshot of current processes; **ps aux** shows all processes for all users in detail.
- **top** (or **htop**) gives a continuously updating LIVE view of CPU/memory usage per process.
- **kill PID** sends a termination signal to a process by its PID; **killall name** kills by process name.

Command Reference

Command	Purpose / Syntax
ps	snapshot of processes in current shell
ps aux	detailed list of ALL processes, all users
top / htop	live, continuously updating process monitor
kill PID	terminate process by PID
killall name	terminate all processes matching a name

Worked Examples

```
$ ps aux | grep apache # find the Apache process and its PID
$ kill 1234 # stop process with PID 1234
```

★ **EXAM TIP:** *Program vs Process: a program is the static file on disk; a process is that program actually RUNNING in memory with a PID.*

■ **COMMON MISTAKE:** kill does not always work instantly on stuck processes — kill -9 PID force-kills, but plain kill sends a polite termination request first.

Chapter 12 — Background Jobs and Job Control

Concept

- Append **&** to a command to run it in the background and get your prompt back immediately.
- **jobs** lists background/suspended jobs in the current shell session.
- **fg** brings a background job to the foreground; **bg** resumes a suspended job IN the background.
- **Ctrl+C** sends SIGINT — it TERMINATES the running foreground process.
- **Ctrl+Z** sends SIGTSTP — it SUSPENDS (pauses) the process, which can later be resumed with fg or bg.

Command Reference

Command	Purpose / Syntax
cmd &	run command in background
jobs	list background/suspended jobs
fg %1	bring job 1 to the foreground
bg %1	resume suspended job 1 in background
Ctrl+C	terminate the current foreground process
Ctrl+Z	suspend (pause) the current foreground process

Worked Examples

```
$ ./longtask.sh & # runs in background, prompt returns immediately
$ fg %1 # resumes first background job in foreground
```

★ *EXAM TIP: Ctrl+C = KILL. Ctrl+Z = PAUSE (not kill) — the paused job still exists and can be resumed with bg or fg. This distinction is a favourite trick question.*

■ **COMMON MISTAKE:** jobs lists jobs for the CURRENT shell session only — it won't show background processes started in a different terminal.

Chapter 13 — Shell Scripting Basics

Concept

- A shell script is just a sequence of shell commands saved in a file, usually starting with a shebang like `#!/bin/bash`.
- Variables are assigned with **name=value** (no spaces around =) and read with **\$name**.
- **\$()** is command substitution — it runs a command and drops its output into the surrounding text.
- Scripts need execute permission (**chmod +x**) before they can be run as `./script.sh`.

Command Reference

Command	Purpose / Syntax
var=value	assign a variable (no spaces around =)
\$var / \${var}	read a variable's value
\$(cmd)	command substitution — insert cmd's output
chmod +x script.sh	make a script executable
./script.sh	run an executable script in current dir
read var	read user input into a variable

Worked Examples

```
#!/bin/bash
echo "Welcome"
name=$(whoami)
echo "Hello, $name"
```

★ *EXAM TIP: Command substitution `$(...)` is the modern, preferred form; the older backtick syntax ``cmd`` does the same thing but is harder to nest.*

■ **COMMON MISTAKE:** Never put spaces around = in a variable assignment (`x = 5` is WRONG; `x=5` is correct) — this is one of the most common shell scripting errors.

Chapter 14 — Conditional Statements

Concept

- **if / elif / else / fi** provide branching logic; conditions are wrapped in `[ ]` or the more modern `[[ ]]`.
- Integer comparisons: **-eq -ne -lt -le -gt -ge** (equal, not-equal, less-than, less-or-equal, greater-than, greater-or-equal).
- String comparisons: **=** and **!=** for equality/inequality; **-z** tests for an empty string.
- Logical operators **&&** (AND) and **||** (OR) combine conditions.

Command Reference

Command	Purpose / Syntax
if [cond]; then ... fi	basic if block
if...elif...else...fi	multi-branch decision
-eq -ne -lt -gt	integer comparisons
= / !=	string equality / inequality
&& / 	logical AND / OR

Worked Examples

```
if [ $age -gt 18 ]; then
echo "Adult"
fi

if [ "$status" = "ok" ]; then
echo "Success"
else
echo "Fail"
fi
```

★ **EXAM TIP:** *-eq/-lt/-gt are for INTEGERS only; use = and != for STRING comparison — mixing them up is a very common exam trap.*

■ **COMMON MISTAKE:** Forgetting the spaces inside `[ cond ]` (must be `[ $x -eq 1 ]`, not `[ $x -eq 1 ]`) causes a syntax error — spacing around brackets is mandatory.

Chapter 15 — Loops in Shell Scripts

Concept

- **for** iterates over a list of values, files, or a numeric range.
- **while** repeats as long as a condition stays TRUE; **until** repeats as long as a condition stays FALSE.
- **break** exits a loop immediately; **continue** skips to the next iteration.
- Loops can iterate over command output too, e.g. for f in \$(ls); do ... done.

Command Reference

Command	Purpose / Syntax
for i in list; do..done	iterate over a fixed list of values
while [cond]; do..done	repeat while condition is TRUE
until [cond]; do..done	repeat while condition is FALSE
break	exit the loop immediately
continue	skip to the next iteration

Worked Examples

```
for i in 1 2 3; do
echo $i
done

i=1
while [ $i -le 5 ]; do
echo $i
i=$((i+1))
done
```

★ *EXAM TIP: while loops need an explicit counter update (i=\$((i+1))) inside the loop body, or they run forever — a classic “spot the bug” question.*

■ **COMMON MISTAKE:** until is the logical OPPOSITE of while — until runs while the condition is false; don't assume it behaves like while.

Chapter 16 — TCS IPA Important Points

Most-Asked Commands

- **wc -w file** — word count (also -l for lines, -c for characters).
- **grep -o -i "text" file | wc -l** — count occurrences of a word, case-insensitive.
- **sort -r** — sort in reverse order. **sort -u** — sort and remove duplicates.
- **chmod** in both octal (755) and symbolic (u+x) form — be fluent in converting between the two.
- **find -type f -perm 0777** — find files with full permissions for everyone.
- **awk 'BEGIN{FS="|";s=0}{s=s+\$2}END{print s/NR}'** — average of a column with a custom delimiter.

Most Confusing Command Pairs

- **grep** (search content) vs **find** (search filesystem for files).
- **sort** vs **uniq** — **uniq** only removes ADJACENT duplicates; you must sort first.
- **kill** (send signal) vs **killall** (kill by process name, not PID).
- **cd ..** (parent directory) vs **cd -** (previous directory) vs **cd ~** (home directory).
- **rmdir** (empty dirs only) vs **rm -r** (any directory, with contents).

Things Students Forget

- **grep -c** counts MATCHING LINES, not total occurrences on a line.
- **Ctrl+Z** suspends (doesn't kill); the job can be resumed with **fg** or **bg**.
- A script needs **chmod +x** AND a shebang line to run as **./script.sh**.
- **umask** sets the DEFAULT for future files — it does not change existing file permissions.
- **^** anchors to the start of a line in **grep**; **\$** anchors to the end.

Exam Shortcuts & One-Liners

Task	One-liner
Count words in a file	<code>wc -w file</code>
Count occurrences of a word	<code>grep -o -i 'word' file wc -l</code>
Unique sorted lines	<code>sort -u file</code>
2nd column, numeric sort	<code>awk '{print \$2}' file sort -n</code>
Replace text in a file	<code>sed 's/old/new/' file</code>
Find files with full permission	<code>find . -type f -perm 0777</code>
Run script in background	<code>./script.sh &</code>
Watch a log file live	<code>tail -f logfile</code>

Complete Command Summary Table

Command	Purpose	Syntax	Key Options	Exam Importance
pwd	Show current directory	pwd	—	Low
cd	Change directory	cd path	~, ~/, ..	High
ls	List directory contents	ls [opts]	-l -a -la	High
touch	Create empty file / update time	touch file	—	Medium
cp	Copy files/dirs	cp src dst	-r	High
mv	Move / rename	mv src dst	—	High
rm	Delete files/dirs	rm [opts] f	-r -f	High
mkdir/rmdir	Create/remove directory	mkdir/rmdir d	—	Medium
find	Search filesystem	find path -opt	-name -type -perm	High
cat	Show file content	cat [opts] f	-n	High
less/more	Page through file	less f	—	Medium
head/tail	First/last N lines	head/tail -nN f	-f (tail)	High
chmod	Change permissions	chmod mode f	octal/symbolic	High
chown	Change owner	chown user f	—	Medium
umask	Default permission mask	umask	—	Medium
grep	Search text pattern	grep pat f	-i -v -c -n -o -E	Very High
find+perm	Find by permission	find -perm 0777	—	Medium
wc	Count lines/words/chars	wc [opts] f	-l -w -c	High
sort	Sort lines	sort [opts] f	-r -u -k -n	High
uniq	Remove adjacent duplicates	uniq f	—	Medium
cut	Extract fields	cut -d, -fN	-d -f	Medium
awk	Field processing language	awk '{...}' f	FS, \$N, NR	High
sed	Stream editing / replace	sed 's/a/b/' f	—	Medium
tr/rev/tac	Translate/reverse chars/lines	tr/rev/tac	—	Low
> / >>	Redirect output (overwrite/append)	cmd > f	—	High
<	Redirect input	cmd < f	—	Medium
 	Pipe commands together	cmd1 cmd2	—	High
whoami/id/grou ps	User identity info	whoami	—	Medium
sudo/su	Elevated privileges	sudo cmd / su	—	High
ps/top	Process listing/monitor	ps aux / top	—	High
kill/killall	Terminate processes	kill PID	-9	Medium
&/jobs/bg/bg	Background job control	cmd &	—	High
chmod +x	Make script executable	chmod +x s.sh	—	High

Chapter 17 — Comparison Tables

The command pairs most often confused in TCS IPA — study these until instant.

grep vs find

Aspect	grep	find
Searches	File CONTENT (text patterns)	File SYSTEM (names, types, permissions)
Typical use	grep 'error' log.txt	find . -name '*.log'
Output	Matching lines	Matching file paths

cat vs less

Aspect	cat	less
Displays	Entire file at once	One page/screen at a time
Best for	Small files	Large files, logs
Navigation	None	Scroll up/down, search with /

rm vs rmdir

Aspect	rm	rmdir
Works on	Files and (with -r) directories	Only EMPTY directories
Deletes contents	Yes, recursively with -r	No — fails if not empty

ps vs top

Aspect	ps	top
View type	Static snapshot	Live, continuously updating
Typical flags	ps aux	top / htop

cp vs mv

Aspect	cp	mv
Source file after	Still exists (duplicated)	No longer exists (moved/renamed)
Use case	Backups, duplication	Renaming, relocating

head vs tail

Aspect	head	tail
Shows	First N lines (default 10)	Last N lines (default 10)
Live mode	Not applicable	tail -f follows growing file

> vs >>

Aspect	>	>>
Existing content	Overwritten / erased	Preserved, new output appended
Use case	Fresh output file	Logging, accumulating output

Ctrl+C vs Ctrl+Z

Aspect	Ctrl+C	Ctrl+Z
Signal	SIGINT	SIGTSTP

Aspect	Ctrl+C	Ctrl+Z
Effect	Terminates the process	Suspends (pauses) the process
Resume?	No, process is gone	Yes — with fg or bg

Program vs Process

Aspect	Program	Process
What it is	Static file/code on disk	A running instance in memory
Identifier	Filename	PID (Process ID)

Kernel vs Shell

Aspect	Kernel	Shell
Role	Core — manages hardware/resources	Interface — interprets user commands
Visibility	Invisible to the user	Directly interacted with

Absolute vs Relative Path

Aspect	Absolute Path	Relative Path
Starts from	Root / always	Current directory
Example	/home/user/file.txt	../file.txt or ./file.txt

su vs sudo

Aspect	su	sudo
Scope	Switches entire session to another user	Elevates privilege for ONE command
Password needed	Target user's password	Your own password

Chapter 18 — Practice MCQs

120 practice questions in TCS IPA style, mixing single concepts and scenario-based combinations. Attempt each before checking the boxed answer beneath it.

1. Which command counts only the number of words in a file?

- A. `wc -w file`
- B. `wc -words file`
- C. `wc file`
- D. `wc -l file`

✓ Answer: A — `wc -w` counts words specifically; plain 'wc' prints lines, words AND characters together.

2. What is the command to view the content of a file directly on the terminal?

- A. `ls`
- B. `vi`
- C. `cat`
- D. `find`

✓ Answer: C — `cat` prints file content directly; `vi` opens an editor, `ls` lists directory entries.

3. Which command switches the current session to another user?

- A. `su`
- B. `switch`
- C. `login`
- D. `user`

✓ Answer: A — `su` (substitute user) switches the active session to another user account.

4. Which command is used to set an environment variable so child processes inherit it?

- A. `env`
- B. `export`
- C. `set`
- D. `unset`

✓ Answer: B — `export` marks a shell variable for export to the environment of child processes.

5. Which command changes the apparent root directory for a process?

- A. `chroot`
- B. `chmod`
- C. `chown`
- D. `chgrp`

✓ Answer: A — `chroot` changes the root directory context for a process and its children.

6. Which symbol runs a process in the background?

- A. `&`
- B. `/`
- C. `Ctrl+C`
- D. `Ctrl+V`

✓ Answer: A — Appending `&` to a command runs it in the background, returning the prompt immediately.

7. Which grep pattern matches 'pat' only at the START of a line?

- A. `^pat`
- B. `$pat`
- C. `pat$`
- D. `pat^`

✓ Answer: A — `^` anchors a regex match to the beginning of the line.

8. Which command sorts lines of a file in reverse order?

- A. sort
- B. sh
- C. sort -r
- D. st

✓ **Answer: C** — `sort -r` reverses the sort order (descending instead of ascending).

9. Which command changes permissions of files matching emp1, emp2 or emp3?

- A. `chmod u+x emp[1-3]`
- B. `chmod 777 emp`
- C. `chmod u+r??? emp`
- D. `chmod 222 emp?`

✓ **Answer: A** — `emp[1-3]` is a wildcard pattern matching emp1, emp2, emp3 exactly.

10. Which command removes an empty directory?

- A. rd
- B. rmdir
- C. dl dir
- D. rdir

✓ **Answer: B** — `rmdir` removes empty directories; it fails if the directory has contents.

11. Which command prints the count of matches of the word 'unix' (case-insensitive)?

- A. `grep -o -i "unix" myfile | wc -l`
- B. `grep -c unix myfile`
- C. `wc -l myfile`
- D. `grep unix myfile`

✓ **Answer: A** — `grep -o` prints each match on its own line; piping to `wc -l` counts total occurrences (`grep -c` only counts matching LINES).

12. `find -type f -perm 0777 -print` displays:

- A. All files with full rwx for all user levels
- B. All directories with full permissions
- C. Files with minimum permission
- D. Folders with full permission

✓ **Answer: A** — `-type f` restricts to regular files; `-perm 0777` means rwx for owner, group, and others.

13. Which awk script correctly computes the average of column 2 in a '|' delimited file?

- A. `awk 'BEGIN{FS="";s=0}{s=s+$2}END{print s/NR}' file1`
- B. `awk 'BEGIN{FS=""}{s=s+$2}END{print s}' file`
- C. `awk 'BEGIN{FS="|";s=0}{s=s+$2}END{print s/NR}' file1`
- D. `awk 'BEGIN{FS=""}{s=s+$2}END{print s/NR}' file1`

✓ **Answer: C** — FS must be set to the actual delimiter '|', and the average requires dividing the sum by NR (number of records).

14. Which command searches 'Hello Unix' in a file, ignoring case?

- A. `grep "Hello Unix" file`
- B. `grep -i "Hello Unix" file`
- C. `grep -c "Hello Unix" file`
- D. `grep -i file "Hello Unix"`

✓ **Answer: B** — `-i` makes `grep`'s search case-insensitive; option order in D is invalid.

15. Which command removes duplicate lines from a file?

- A. sort -u
- B. sort uniq
- C. uniq
- D. unique

✓ Answer: A — sort -u sorts and removes duplicates in one step; plain 'uniq' only removes ADJACENT duplicates, so input must already be sorted.

16. Which directory holds essential system configuration files?

- A. /bin
- B. /etc
- C. /var
- D. /dev

✓ Answer: B — /etc is the standard location for system-wide configuration files, e.g. /etc/hosts.

17. Which directory holds device files in a Unix system?

- A. /dev
- B. /tmp
- C. /usr
- D. /opt

✓ Answer: A — /dev holds device files such as /dev/sda, reflecting the 'everything is a file' philosophy.

18. A hidden file in Unix is one whose name:

- A. Starts with a dot (.)
- B. Ends with .hid
- C. Has no extension
- D. Is stored in /tmp

✓ Answer: A — Filenames beginning with a dot are treated as hidden and excluded from a plain ls.

19. Which command shows your current directory path?

- A. cd
- B. pwd
- C. ls
- D. dir

✓ Answer: B — pwd = print working directory.

20. cd - moves you to:

- A. The root directory
- B. The parent directory
- C. Your home directory
- D. The previous directory you were in

✓ Answer: D — cd - toggles back to whichever directory you were in before your last cd command.

21. Which option to ls shows hidden files along with a detailed listing?

- A. ls -x
- B. ls -la
- C. ls -h
- D. ls -r

✓ Answer: B — ls -l gives long/detailed format, -a includes hidden files; combined as -la.

22. Which command creates an empty file or updates its timestamp?

- A. touch
- B. mkdir
- C. new
- D. vi

✓ Answer: A — touch creates a file if absent, or updates the modification time if it exists.

23. Which command permanently deletes a directory and all its contents without confirmation?

- A. rm -r dir
- B. rm -rf dir
- C. rmdir dir
- D. del dir

✓ Answer: B — rm -rf forces recursive deletion without any confirmation prompts.

24. rmdir will fail if the target directory:

- A. Is hidden
- B. Is not empty
- C. Has read permission
- D. Is owned by root

✓ Answer: B — rmdir only removes EMPTY directories; use rm -r for non-empty ones.

25. Which command displays only the first 5 lines of a file?

- A. head -n5 file
- B. top -5 file
- C. head file 5
- D. first -5 file

✓ Answer: A — head -n5 (or head -5) restricts output to the first 5 lines.

26. Which command lets you watch a log file update in real time?

- A. cat file
- B. tail -f file
- C. head -f file
- D. less -f file

✓ Answer: B — tail -f continuously displays new lines appended to a growing file.

27. Which paging command supports scrolling both forward AND backward?

- A. more
- B. cat
- C. less
- D. head

✓ Answer: C — less supports bi-directional scrolling and searching; more is forward-only.

28. What does chmod 644 set for a file?

- A. rwxr-xr-x
- B. rw-r--r--
- C. rwxrwxrwx
- D. r--r--r--

✓ Answer: B — 6=rw-, 4=r--, 4=r-- — owner can read/write, group and others can only read.

29. What does `chmod 755` set for a file?

- A. `rwXr-Xr-X`
- B. `rw-r--r--`
- C. `rwX-----`
- D. `rwXrwxrwx`

✓ Answer: A — 7=rwx (owner), 5=r-x (group), 5=r-x (others).

30. Which command changes a file's owner?

- A. `chmod`
- B. `chown`
- C. `chgrp`
- D. `umask`

✓ Answer: B — `chown` changes the owning user (and optionally group) of a file.

31. Which command sets the default permission mask for newly created files?

- A. `chmod`
- B. `umask`
- C. `chown`
- D. `setperm`

✓ Answer: B — `umask` subtracts from the maximum default permissions to set defaults for new files.

32. Which `chmod` symbolic operator ADDS a permission without removing others?

- A. `+`
- B. `-`
- C. `=`
- D. `!`

✓ Answer: A — `+` adds the specified permission; `-` removes it; `=` sets exact permissions, clearing others.

33. `grep -v "error"` file will display:

- A. Only lines containing 'error'
- B. Only lines NOT containing 'error'
- C. Count of 'error' lines
- D. Nothing

✓ Answer: B — `-v` inverts the match, printing lines that do NOT contain the pattern.

34. Which command extracts the first field of a comma-delimited file?

- A. `cut -d',' -f1 file`
- B. `cut -f1 -d, file1`
- C. `awk -F, '{print $1}' file`
- D. Both A and C are correct

✓ Answer: D — `cut -d',' -f1` and `awk -F',' '{print $1}'` both extract the first comma-delimited field.

35. `uniq` removes duplicate lines only if they are:

- A. Anywhere in the file
- B. Adjacent to each other
- C. In the first column
- D. Numeric

✓ Answer: B — `uniq` compares consecutive lines only — sort the file first for it to catch all duplicates.

36. Which sed command replaces 'orange' with 'blue' in a file?

- A. sed 's/orange/blue/' file
- B. sed 'orange=blue' file
- C. sed replace orange blue file
- D. sed -r orange blue file

✓ Answer: A — sed's substitute syntax is s/pattern/replacement/.

37. Which command reverses the characters within each line of text?

- A. tac
- B. rev
- C. sort -r
- D. cut -r

✓ Answer: B — rev reverses the character order within each line; tac reverses the ORDER of lines instead.

38. Which command reverses the ORDER of lines/files (like 'cat' backwards)?

- A. rev
- B. tac
- C. sort -r
- D. tail -r

✓ Answer: B — tac (cat spelled backwards) prints lines in reverse order, last line first.

39. What does > do when the target file already exists?

- A. Appends to it
- B. Overwrites it completely
- C. Refuses to run
- D. Prompts for confirmation

✓ Answer: B — > always overwrites/truncates an existing file; use >> to append instead.

40. Which symbol feeds a file's content as input to a command?

- A. >
- B. >>
- C. <
- D. |

✓ Answer: C — < redirects a file's content to be used as a command's standard input.

41. echo "hello" | wc counts:

- A. Lines, words, and characters
- B. Only characters
- C. Only words
- D. Nothing, echo cannot be piped

✓ Answer: A — Plain wc (no flags) prints line count, word count, and character count together.

42. Which command shows the current logged-in username?

- A. whoami
- B. id
- C. groups
- D. user

✓ Answer: A — whoami prints just the current username.

43. Which command lists all the groups a user belongs to?

- A. whoami
- B. id
- C. groups
- D. sudo

✓ Answer: C — groups lists group memberships for a given (or current) user.

44. sudo differs from su because it:

- A. Requires the target user's password
- B. Elevates privilege for a single command only
- C. Cannot run as root
- D. Only works on macOS

✓ Answer: B — sudo runs one command with elevated rights using YOUR OWN password; su switches the whole session.

45. Which shell is most widely used and is the Bash acronym expansion of?

- A. Bourne Again Shell
- B. Basic Shell
- C. Bourne Alternate Shell
- D. Binary Shell

✓ Answer: A — bash = Bourne Again SHell, an enhanced version of the original Bourne shell (sh).

46. Which line at the top of a script specifies the interpreter to use?

- A. #comment
- B. #!/bin/bash
- C. //bash
- D.

✓ Answer: B — This is the shebang line; the OS uses it to determine which interpreter runs the script.

47. A process is best described as:

- A. A file stored on disk
- B. A running instance of a program
- C. A type of shell
- D. A permission set

✓ Answer: B — A process is a program actively executing in memory, identified by a unique PID.

48. Which command gives a continuously updating live view of running processes?

- A. ps
- B. top
- C. jobs
- D. find

✓ Answer: B — top refreshes periodically to show live CPU/memory usage per process.

49. Which command terminates a process using its PID?

- A. kill PID
- B. stop PID
- C. end PID
- D. ps -k PID

✓ Answer: A — kill sends a termination signal to the process with the given PID.

50. Ctrl+Z does which of the following to a running foreground process?

- A. Terminates it permanently
- B. Suspends (pauses) it
- C. Restarts it
- D. Sends it to background automatically running

✓ Answer: B — Ctrl+Z sends SIGTSTP, pausing the process; it can be resumed later with fg or bg.

51. Which command resumes a suspended job and brings it to the foreground?

- A. bg
- B. fg
- C. jobs
- D. resume

✓ Answer: B — fg brings a background/suspended job back into the foreground.

52. Which command lists the background/suspended jobs of the current shell?

- A. ps
- B. jobs
- C. top
- D. bg

✓ Answer: B — jobs lists jobs associated specifically with the current shell session.

53. A shell script variable is assigned correctly with:

- A. x = 5
- B. x=5
- C. \$x=5
- D. var x=5

✓ Answer: B — No spaces are allowed around = in a shell variable assignment; 'x = 5' causes a syntax error.

54. Which syntax performs command substitution in a shell script?

- A. #(cmd)
- B. \$(cmd)
- C. %(cmd)
- D. &(cmd)

✓ Answer: B — \$(cmd) executes cmd and substitutes its output into the surrounding command/string.

55. Which comparison operator checks integer equality in an if condition?

- A. ==
- B. -eq
- C. =
- D. eq

✓ Answer: B — -eq is the correct test operator for integer equality inside [] conditions in shell scripts.

56. Which comparison operator checks STRING equality in an if condition?

- A. -eq
- B. =
- C. -eq or =
- D. equals

✓ Answer: B — String equality uses = (or == in some shells), never -eq which is reserved for integers.

57. Which loop type repeats while a condition remains FALSE?

- A. for
- B. while
- C. until
- D. loop

✓ Answer: C — until is the logical opposite of while — it keeps looping until the condition becomes true.

58. Which statement immediately exits a loop before it completes naturally?

- A. continue
- B. break
- C. exit
- D. stop

✓ Answer: B — break exits the loop entirely; continue only skips to the next iteration.

59. Scenario: You need to find how many times 'error' appears (not lines, total occurrences) in a log. Best command?

- A. `grep -c error log`
- B. `grep -o error log | wc -l`
- C. `wc -l log`
- D. `grep error log`

✓ Answer: B — `grep -c` counts matching LINES; to count total OCCURRENCES you must extract each match with `-o` and count them.

60. Scenario: You want to make all .sh files in a folder executable for the owner only. Best command?

- A. `chmod 777 *.sh`
- B. `chmod u+x *.sh`
- C. `chmod a+x *.sh`
- D. `chown u+x *.sh`

✓ Answer: B — `u+x` adds execute permission for the owner only, without touching group/other or read/write bits.

61. Scenario: You started a long backup script and want your terminal back immediately. What do you type?

- A. `backup.sh &`
- B. `backup.sh !`
- C. `bg backup.sh`
- D. `fg backup.sh`

✓ Answer: A — Appending `&` runs the command in the background and returns control of the terminal immediately.

62. Scenario: A script paused with `Ctrl+Z` needs to keep running but in the background. What next?

- A. `fg`
- B. `bg`
- C. `kill`
- D. `jobs`

✓ Answer: B — `bg` resumes a suspended job but keeps it running in the background, freeing your terminal.

63. Scenario: You need unique, sorted email addresses from a messy list file. Best one-liner?

- A. `sort emails.txt`
- B. `uniq emails.txt`
- C. `sort -u emails.txt`
- D. `grep emails.txt`

✓ Answer: C — `sort -u` sorts the file AND removes duplicates in a single step, correctly handling non-adjacent duplicates.

64. Scenario: You must give a script rwx to owner, r-x to group, and no access to others. Correct chmod?

- A. chmod 750 script.sh
- B. chmod 705 script.sh
- C. chmod 570 script.sh
- D. chmod 057 script.sh

✓ Answer: A — 7=rwx (owner), 5=r-x (group), 0=--- (others) — in that positional order: owner, group, other.

65. Scenario: You need to view only NEW lines being written to a growing log file. Command?

- A. cat log
- B. head log
- C. tail -f log
- D. less log

✓ Answer: C — tail -f is designed exactly for this — live-following a growing file.

66. Scenario: You want to append today's date to an existing report file without erasing it. Command?

- A. date > report.txt
- B. date >> report.txt
- C. date < report.txt
- D. date | report.txt

✓ Answer: B — >> appends to the end of the file, preserving existing content; > would erase it.

67. Which command displays disk usage of a directory in human-readable form?

- A. du -h
- B. df -h
- C. free -h
- D. top

✓ Answer: A — du -h reports disk usage of files/directories in human-readable sizes (K, M, G).

68. Which command displays available and used disk space per filesystem?

- A. du -h
- B. df -h
- C. free -h
- D. ps aux

✓ Answer: B — df -h shows filesystem-level disk space usage in human-readable form.

69. Which command shows memory (RAM) usage?

- A. free -h
- B. df -h
- C. du -h
- D. top -m

✓ Answer: A — free -h reports total, used, and free memory in human-readable units.

70. Which command creates a compressed .tar.gz archive?

- A. tar cvf archive.tar files
- B. tar caf archive.tar.gz files
- C. zip archive.tar.gz files
- D. gzip archive.tar files

✓ Answer: B — tar caf with a .tar.gz filename auto-detects gzip compression while archiving.

71. Which command extracts a .zip archive?

- A. tar xf file.zip
- B. unzip file.zip
- C. gunzip file.zip
- D. untar file.zip

✓ Answer: B — unzip is the dedicated tool for extracting .zip archives.

72. Which command downloads a file directly from a URL?

- A. curl / wget
- B. ftp
- C. scp
- D. rsync

✓ Answer: A — Both curl and wget can fetch files over HTTP/HTTPS from a URL.

73. Which command displays the last 20 commands you typed?

- A. history
- B. log
- C. past
- D. cmdlist

✓ Answer: A — history shows your recent command history, by default the last several hundred entries.

74. Which command creates a shortcut/alias for a longer command?

- A. alias
- B. shortcut
- C. link
- D. ref

✓ Answer: A — alias lets you define a short name that expands to a longer command.

75. Which permission digit represents read + write but NOT execute?

- A. 7
- B. 6
- C. 5
- D. 4

✓ Answer: B — $6 = 4(r) + 2(w) + 0(x)$ = read and write only.

76. Which permission digit represents read-only access?

- A. 1
- B. 2
- C. 4
- D. 5

✓ Answer: C — $4 =$ read only (r--).

77. In octal notation, what does 0 represent?

- A. Full access
- B. No access at all
- C. Read only
- D. Execute only

✓ Answer: B — $0 = ---$, no permissions granted at all.

78. Which command displays a file's permissions, owner, and size?

- A. ls
- B. ls -l
- C. cat -l
- D. stat -s

✓ Answer: B — ls -l gives the long-format listing including permissions, owner, group, size, and date.

79. Which grep option displays the LINE NUMBER of each match?

- A. -n
- B. -c
- C. -l
- D. -o

✓ Answer: A — -n prefixes each matching line with its line number in the file.

80. Which grep option restricts matching to EXTENDED regular expressions (e.g. {n,}, |)?

- A. -E
- B. -x
- C. -r
- D. -P

✓ Answer: A — -E enables extended regex syntax such as {n,} for repetition and | for alternation.

81. What is the correct regex to match words with 6 or more letters using grep -E?

- A. \w{6,}
- B. \w{6}
- C. \w+6
- D. {6,}\w

✓ Answer: A — \w{6,} matches 6 or more consecutive word characters.

82. Which awk built-in variable represents the total number of records (rows) processed?

- A. NF
- B. NR
- C. FS
- D. RS

✓ Answer: B — NR = Number of Records processed so far; NF = Number of Fields in the current record.

83. Which awk built-in variable represents the number of fields in the current record?

- A. NF
- B. NR
- C. FS
- D. OFS

✓ Answer: A — NF holds the count of fields (columns) in the current line being processed.

84. Which awk variable sets the input field separator?

- A. FS
- B. OFS
- C. RS
- D. ORS

✓ Answer: A — FS (Field Separator) defines how awk splits each input line into fields.

85. Which command combination finds and deletes all .tmp files in the current tree?

- A. `find . -name '*.tmp' -delete`
- B. `rm *.tmp`
- C. `find *.tmp -rm`
- D. `delete . -name *.tmp`

✓ Answer: A — `find` with `-delete` locates matching files recursively and removes them directly.

86. Which command shows only DIRECTORIES using `find`?

- A. `find . -type d`
- B. `find . -type f`
- C. `find . -dir`
- D. `find . -directory`

✓ Answer: A — `-type d` restricts `find`'s results to directories only.

87. A relative path such as `../data/file.txt` refers to a location:

- A. From the root directory
- B. From the current directory's parent
- C. From the home directory always
- D. From `/tmp`

✓ Answer: B — `../` moves up one level from the current directory before descending into `data/`.

88. Which of these is an ABSOLUTE path?

- A. `reports/2024/data.csv`
- B. `./data.csv`
- C. `/home/user/data.csv`
- D. `../data.csv`

✓ Answer: C — Only paths starting with `/` (the root) are absolute; the others are relative to the current directory.

89. Tab completion in the shell is primarily used to:

- A. Auto-complete file/command names
- B. Switch users
- C. Kill a process
- D. Clear the terminal

✓ Answer: A — Pressing `Tab` auto-completes partially typed file, directory, or command names.

90. Which wildcard matches exactly ONE character in a filename?

- A. `*`
- B. `?`
- C. `[]`
- D. `%`

✓ Answer: B — `?` matches exactly one character; `*` matches any number of characters (including zero).

91. Which wildcard pattern matches a single character from a defined set?

- A. `*`
- B. `?`
- C. `[abc]`
- D. `{abc}`

✓ Answer: C — `[abc]` matches exactly one character that is either `a`, `b`, or `c`.

92. What does the pipe (|) operator fundamentally do?

- A. Writes output to a file
- B. Redirects input from a file
- C. Sends stdout of one command into stdin of the next
- D. Deletes a file

✓ Answer: C — Pipes chain commands together, letting one command's output become the next command's input.

93. Which command counts the number of LINES in a file?

- A. wc -l
- B. wc -w
- C. wc -c
- D. wc -n

✓ Answer: A — wc -l reports the line count of the given file.

94. Which command counts the number of CHARACTERS in a file?

- A. wc -c
- B. wc -w
- C. wc -l
- D. wc -n

✓ Answer: A — wc -c reports the byte/character count of the file.

95. Which is the correct symbolic form to give a file read and write for the group only?

- A. chmod g=rw file
- B. chmod g+rw file
- C. chmod o=rw file
- D. chmod u=rw file

✓ Answer: A — g=rw sets the GROUP's permission to exactly read+write, replacing whatever it had before.

96. Which command displays only lines 1 through 5 of a file?

- A. head -n5 file
- B. tail -n5 file
- C. cat -5 file
- D. sed -n '1,5p' file

✓ Answer: D — sed -n '1,5p' prints lines 1 to 5; head -n5 also gives the same result for the FIRST 5 lines specifically.

97. Which environment variable typically stores the path to the current shell?

- A. \$SHELL
- B. \$PATH
- C. \$HOME
- D. \$USER

✓ Answer: A — \$SHELL holds the path of the user's default/current shell, e.g. /bin/bash.

98. Which variable holds a list of directories the shell searches for executables?

- A. \$PATH
- B. \$SHELL
- C. \$HOME
- D. \$PWD

✓ Answer: A — \$PATH is a colon-separated list of directories searched when you type a command name.

99. Which command displays the current value of all shell environment variables?

- A. env
- B. set
- C. printenv
- D. Both A and C

✓ Answer: D — Both env and printenv list current environment variables; 'set' shows shell variables including local ones.

100. What happens if you run 'kill' on a PID that doesn't exist?

- A. It silently succeeds
- B. It returns a 'no such process' error
- C. It creates a new process
- D. The system reboots

✓ Answer: B — kill reports an error such as 'No such process' when given an invalid PID.

101. Which signal does kill send by default (no flag specified)?

- A. SIGTERM
- B. SIGKILL
- C. SIGSTOP
- D. SIGINT

✓ Answer: A — Plain kill sends SIGTERM (signal 15), a polite termination request the process can catch and clean up after.

102. Which flag forces an immediate, un-catchable process kill?

- A. kill -1
- B. kill -9
- C. kill -15
- D. kill -0

✓ Answer: B — kill -9 sends SIGKILL, which the OS enforces immediately and cannot be intercepted by the process.

103. Which shell feature lets you run two conditions where the SECOND only executes if the FIRST succeeds?

- A. ||
- B. &&
- C. ;
- D. |

✓ Answer: B — && is logical AND — the second command runs only if the first exits successfully (status 0).

104. Which shell feature runs the second command only if the FIRST one FAILS?

- A. &&
- B. ||
- C. ;
- D. &

✓ Answer: B — || is logical OR — the second command runs only if the first one returns a non-zero (failure) exit status.

105. ';' between two commands in the shell means:

- A. Run both, one after another regardless of success
- B. Run the second only if the first succeeds
- C. Pipe the first into the second
- D. Run both in parallel

✓ Answer: A — ; simply sequences commands one after another with no dependency on exit status.

106. Which command displays a manual page for another command?

- A. man cmd
- B. help cmd
- C. info cmd only
- D. --help only

✓ **Answer: A** — man opens the manual page documentation for the given command.

107. Scenario: grep 'test' file1 file2 is run. What extra info appears in the output vs. searching a single file?

- A. Line numbers always
- B. The filename is prefixed to each matching line
- C. Nothing changes
- D. Only file2 is searched

✓ **Answer: B** — When grep searches multiple files, it automatically prefixes each output line with the source filename.

108. Which command displays the type of a file (text, binary, directory, etc.)?

- A. file filename
- B. type filename
- C. stat filename
- D. info filename

✓ **Answer: A** — The 'file' command inspects and reports the type/format of the given file.

109. Which is true about symbolic links (created with ln -s)?

- A. They are copies of the original file
- B. They are pointers/references to another file
- C. They double disk usage
- D. They cannot be deleted

✓ **Answer: B** — A symbolic link is a lightweight pointer to another file/path, not a duplicate of its content.

110. Which command creates a symbolic link named 'link' pointing to 'target'?

- A. ln -s target link
- B. ln target link
- C. link target -s
- D. ls -s target link

✓ **Answer: A** — ln -s source link_name creates a symbolic (soft) link.

111. A hard link, unlike a symbolic link:

- A. Can span across different filesystems freely
- B. Points directly to the same inode as the original file
- C. Can link to directories easily
- D. Breaks if the original file is renamed

✓ **Answer: B** — A hard link shares the same inode/data as the original; deleting the original name doesn't remove the data while a hard link remains.

112. Which command displays the exit status of the last executed command?

- A. echo \$?
- B. echo \$!
- C. echo \$\$
- D. echo \$0

✓ **Answer: A** — \$? holds the exit status (return code) of the most recently executed command; 0 means success.

113. Which special variable holds the PID of the last background process?

- A. \$?
- B. \$!
- C. \$\$
- D. \$#

✓ Answer: B — \$! stores the PID of the most recently started background job.

114. Which special variable holds the PID of the current shell itself?

- A. \$?
- B. \$!
- C. \$\$
- D. \$0

✓ Answer: C — \$\$ expands to the PID of the currently running shell/script.

115. Which command safely tests if a file exists before acting on it in a script?

- A. if [-f file]; then ... fi
- B. if file exists; then ... fi
- C. if -f file then ... fi
- D. check file

✓ Answer: A — [-f file] is the standard test for 'file exists and is a regular file' inside a shell condition.

116. Which test flag checks if a path is a directory?

- A. -f
- B. -d
- C. -e
- D. -x

✓ Answer: B — -d tests whether the given path exists AND is a directory.

117. Which test flag simply checks that a path exists, regardless of type?

- A. -e
- B. -f
- C. -d
- D. -x

✓ Answer: A — -e is true if the path exists at all, whether it's a file, directory, or link.

118. A for loop 'for f in *.txt; do echo \$f; done' will:

- A. Print each .txt filename in the current directory
- B. Print the literal string *.txt
- C. Fail with a syntax error
- D. Delete all .txt files

✓ Answer: A — The shell expands *.txt into the matching filenames before the loop assigns them one by one to \$f.

119. What is wrong with this loop: while [\$i -le 5]; do echo \$i; done (no update to i)?

- A. Nothing, it works fine
- B. It becomes an infinite loop since i never changes
- C. It only runs once
- D. It throws a syntax error

✓ Answer: B — Without incrementing i inside the loop body, the condition \$i -le 5 never becomes false, so the loop never terminates.

120. Which command duplicates an entire directory tree, including subfolders?

- A. `cp file dest`
- B. `cp -r dir dest`
- C. `mv dir dest`
- D. `cp -x dir dest`

✓ **Answer: B** — `cp -r` recursively copies a directory and everything inside it.

121. Which shell construct allows multi-way branching based on a variable's value (like a switch statement)?

- A. `if/elif`
- B. `case`
- C. `for`
- D. `select`

✓ **Answer: B** — `case ... esac` provides pattern-based multi-way branching, similar to switch statements in other languages.

122. Scenario: You want to list only files larger than 10MB in the current directory tree. Best approach?

- A. `find . -size +10M`
- B. `ls -la | grep 10M`
- C. `du -h | grep 10`
- D. `find . -name '10M'`

✓ **Answer: A** — `find's -size` option with `+10M` finds files larger than 10 megabytes.

123. Scenario: You need to change ONLY the group ownership of a file, not the user owner. Best command?

- A. `chown :groupname file`
- B. `chown groupname file`
- C. `chgrp -u groupname file`
- D. `chmod groupname file`

✓ **Answer: A** — `chown :groupname file` (with the leading colon) changes only the group, leaving the user owner unchanged (`chgrp` works too, but wasn't offered correctly here).

124. Scenario: A colleague asks the difference between '`sh script.sh`' and '`./script.sh`'. What's correct?

- A. They are always identical in every case
- B. `sh script.sh` runs it under the `sh` interpreter regardless of the shebang; `./script.sh` uses the shebang-specified interpreter
- C. `./script.sh` doesn't require execute permission
- D. `sh script.sh` requires `chmod +x` but `./script.sh` doesn't

✓ **Answer: B** — Running via '`sh script.sh`' forces the `sh` interpreter and ignores the shebang; '`./script.sh`' relies on the shebang line and needs execute permission.

Chapter 19 — Rapid Revision — Final Hours

Read only this section in the last hour before the exam. It condenses the entire handbook.

Core Concepts in One Line Each

- **Kernel** = manages hardware. **Shell** = interprets your commands. **Process** = running program with a PID.
- **/etc** configs, **/dev** devices, **/home** or **/Users** user data, **/tmp** temp files, **/var** logs.
- Absolute path starts with **/**, always the same; relative path depends on your current directory.
- Everything in Unix, including devices, is treated as a file.

Command Firing Order

Category	Commands to recall instantly
Navigate	pwd, cd, ls -la
Create/Manage	touch, mkdir, cp, mv, rm -rf
View	cat -n, head/tail -n, tail -f, less
Permissions	chmod (octal/symbolic), chown, umask
Search/Filter	grep -i -v -c -n -o -E, find -type -perm, sort -r -u -k, uniq, cut, awk, sed
Redirect	> overwrite, >> append, < input, pipe
Users	whoami, id, groups, sudo, su, passwd
Processes/Jobs	ps aux, top, kill, &, jobs, fg, bg, Ctrl+C, Ctrl+Z
Scripting	#!/bin/bash, \$var, \$(cmd), if/elif/else, for/while/until, chmod +x

The 12 Comparisons You Must Not Confuse

- grep (content) vs find (files/filesystem)
- sort vs uniq (uniq only removes ADJACENT duplicates)
- cp (keeps source) vs mv (removes source)
- rm -r (recursive delete) vs rmdir (empty dirs only)
- ps (snapshot) vs top (live view)
- head vs tail (first vs last N lines); tail -f follows live
- > (overwrite) vs >> (append)
- Ctrl+C (kill) vs Ctrl+Z (suspend, resumable)
- su (switch user, full session) vs sudo (one command, elevated)
- Program (static file) vs Process (running instance with PID)
- kernel (manages hardware) vs shell (interprets commands)
- while (loops if condition TRUE) vs until (loops if condition FALSE)

Octal Permissions — Memorise This Grid

Value	Meaning
7 = rwx	4(r) + 2(w) + 1(x) — full access
6 = rw-	4(r) + 2(w) — read & write, no execute
5 = r-x	4(r) + 1(x) — read & execute, no write
4 = r--	read only
0 = ---	no access at all

Last-Minute Exam Tips

- If a question mixes two commands (e.g. `grep + wc`, `find + chmod`), solve LEFT TO RIGHT — apply the first command's output as input to the second.
- When an option looks unfamiliar, eliminate answers that contradict a rule you know for certain (e.g. `rmdir` never deletes non-empty folders).
- For `awk/sed` questions, check the delimiter (FS) and quoting first — most wrong options break syntax subtly.
- `grep -c` counts LINES, not occurrences — this single fact resolves several question variants.
- Read `chmod` questions carefully: the order is always User, Group, Others.

All the best for your TCS IPA exam.